SENTINEL Lab Manual

The facts for all four labs

ECE 476 Engineering System Software • Fall 2026 • University of Victoria

This manual gives the facts that apply to each lab. The course Lab Documentation[1] describes the scenario: the detachment, the four sensors, the four parts of SENTINEL, and a user story. `README.md`, at the top of your repository, lists each document of the course and where it is.

1 SENTINEL

SENTINEL is the software of a surveillance operations centre. Four sensors send contact reports to the centre. SENTINEL collects the reports, records each action in the mission log, sorts the reports by importance, and keeps one correct picture of the contacts. The mission log is a file: each line is one action, with its time. You write it in C, for Linux, as four libraries. Each lab adds one library, and each library is a layer on the library before it.

1.1 The four labs

Lab	Session	Library	What it adds
1	Oct 9	`libsyscall`	The interface to the operating system: the feed, the mission log, and precise timestamps
2	Oct 30	`libproc`	One process for each sensor, and the channels that bring their reports to the centre
3	Nov 20	`libipc`	Worker threads that share one processor and keep the track picture
4	Dec 4	`libsync`	Protection for each thing that the workers share

The code of each lab is in `src/<library>/`, for example `src/libsyscall/`. In Lab 4, `src/sentinel/` also starts all four layers with one command. The Submission Guide[2] gives the deadline and the marks of each lab.

1.2 The six requirements

The documents name a requirement by its ID.

[1] The ECE 476 Lab Documentation for SENTINEL. The course gives it to you. It describes the detachment, the four sensors, the four parts of SENTINEL, and a user story.

[2] The Submission Guide, `docs/submit/submit.pdf` in your repository. It gives the marks of each lab, the oral exam, and the steps of the hand-in.

ID	Requirement	Lab
R1	Four sensors run at very different rates, and a sensor can stop with no warning	2
R2	An urgent contact must overtake routine traffic	2
R3	Thousands of contacts arrive each second	3
R4	Many workers write the same shared record, and must not corrupt it	4
R5	The centre must never freeze while it still looks alive	4
R6	Each action has a timestamp, and the centre writes it to the mission log	1

1.3 The contact report

A contact report is one line of text:

```
1755534061.482913 NWS ACCF-4587 68.7100 -105.7000 420.0 P
```

`common/contact.h` states each field, the limit of 128 bytes, and the code and the rate of each sensor. `NWS`, `AUR` and `AIS` send at their rates for the whole run. The satellite, `SAT`, sends in passes: 2000 reports each second for 10 s, then no report for 10 s, and then the next pass. Its process continues in the gap, so a quiet `SAT` has not stopped. All four labs use this format, so do not change it.

1.4 The two priorities

The last field of a report is the **flag of the sensor**. `P` means that the sensor marked the report urgent, and `R` means that the report is routine. A sensor marks a report urgent because of an event that it saw, for example an emergency code from a transponder. The flag belongs to one report, not to a contact. `contactfeed` (Section 3.1) sets the flag `P` on one report in ten.

A sensor cannot know if the centre expects a contact, because only the centre has the list of filed traffic, `common/traffic.txt`. So the centre has a priority of its own, **HIGH**: a contact that is on no list, in a region that the centre monitors. From Lab 2, the centre handles HIGH reports first, then reports with the flag `P`, then routine reports. Lab 1 only records the flag.

1.5 One processor

The operations centre runs on one processor. A thread is a separate path through a program, and on one processor only one thread runs at a time. The operating system changes the thread that runs every few milliseconds, and each change is a context switch.

Term	Meaning	Processors
Concurrency	Many tasks are in progress at the same time	One is enough
Parallelism	Many tasks execute at the same instant	Many

So eight worker threads on one processor do not do more work than one worker. They give overlap. While one worker waits for a lock, a file or the queue, a different worker uses the processor. Also, the centre can start an urgent report while routine work continues. In Lab 3, the page (Section 5) shows the share of a processor that all the workers got together. On one processor, that share cannot be more than 1.00.

Two given functions in `common/cpu.h` make and measure this condition. `cpu_pin` keeps a program on a set of processors, and `cpu_thread_ns` counts the processor time that a thread got.

2 The repository

2.1 What is in it

```
sentinel-lab/
  Makefile          the top-level build. GIVEN
  README.md         the list of documents. GIVEN
  src/              YOUR CODE: one directory for each lab
    libsyscall/       Lab 1
    libproc/          Lab 2
    libipc/           Lab 3
    libsync/          Lab 4
    sentinel/         Lab 4: the complete system
  common/           the given code. GIVEN
  tools/accept/     the source of make accept1, accept2 and accept3. GIVEN
  tools/bin/        the given programs, for aarch64 and for x86_64. GIVEN
  web/              the page. GIVEN
  docs/
    DEFECTS.md        your record of faults
    manual/           this manual
    setup/            the Setup Guide
    submit/           the Submission Guide
    lab1/             the papers of Lab 1, and the three files of your Lab 1 run
    lab2/             the papers of Lab 2, and the three files of your Lab 2 hand-in
    lab3/             the papers of Lab 3, and the three files of your Lab 3 hand-in
  build/            the files that make builds. Git ignores it
```

`make` creates `build/`, and it puts each file that it makes there. It also copies the given programs for your processor from `tools/bin/` into `build/`. The Setup Guide[3] tells you how to make your own copy of the repository, and the Submission Guide[4] tells you how to hand in your work.

2.2 Who owns which files

Files	Who changes them
`src/<lab>/src`, `src/<lab>/tests`, `src/libsyscall/Makefile`	you
`src/<lab>/include`	the course. Do not change a declaration
`src/<lab>/given`, and the `Makefile` of each lab from Lab 2	the course
`docs/DEFECTS.md`, and the files of your run in `docs/<lab>/`	you
`common/`, `tools/`, `web/`, the papers in `docs/`, the top `Makefile`, each `README.md`	the course

Obey this table, and `git pull upstream main` has no conflict. If you change a file that the course owns, the pull stops and reports a conflict. Then take the copy of the course:

```
git checkout --theirs -- <the file>
```

[3] The Setup Guide, `docs/setup/setup.pdf` in your repository. It sets up your machine and your own copy of the repository.

[4] The Submission Guide, `docs/submit/submit.pdf` in your repository. It gives the marks of each lab, the oral exam, and the steps of the hand-in.

```
git add <the file>
git commit
```

Do the same for a given program in `tools/bin/`. A program is a binary file, and Git cannot merge a binary file.

2.3 The given code

Header	What it is for
`common/contact.h`	The contact record, and `contact_parse`, which makes a record from one line
`common/linereader.h`	A reader that returns whole lines from a file, a pipe or a device
`common/cpu.h`	The processors that a program can use, and the processor time of a thread
`common/device.h`	Opens a serial device
`common/traffic.h`	The filed traffic and the monitored regions, from `common/traffic.txt`

Each header states the contract of each function that it declares. The given code compiles without `-Werror`, so a warning from a newer compiler cannot stop your build. Your code compiles with `-Wall -Wextra -Werror`, so a warning stops the build.

3 The given programs

`make` copies four given programs into `build/`: `contactfeed`, `benchfeed`, `benchsim` and `benchd`. Run each one with `--help` to see its usage.

3.1 `contactfeed`

`contactfeed` writes contact reports on its standard output, in place of the four sensors.

```
# 20 reports each second, until Ctrl-C
build/contactfeed --sensor NWS --rate 20
# passes: 2000 reports each second for 10 s, then none for 10 s
build/contactfeed --sensor SAT
# 2000 reports each second with no gap, until Ctrl-C
build/contactfeed --sensor SAT --pass 0
# 50 reports at the rate of AIS
build/contactfeed --sensor AIS --count 50
# the same 20 contacts each time
build/contactfeed --sensor AUR --burst 20 --seed 42
# a feed file of 500 reports
build/contactfeed --sensor NWS --burst 500 > feed.txt
```

Option	What it does
`--sensor NWS|AUR|AIS|SAT`	The sensor. This option is necessary
`--rate R`	R reports each second. With no `--count`, it does not stop
`--burst N`	N reports as fast as possible, then it stops
`--count N`	It stops after N reports
`--priority-every K`	Each Kth report has the flag `P`. The default is 10
`--seed S`	The same seed gives the same contacts
`--profile NAME`	`ideal`, `world`, `dirty`, `field` or `threat`
`--damage-every N`	With `dirty`, `contactfeed` damages each Nth line. The default is 50
`--pass ON,OFF`	For `SAT` only: ON seconds with reports, then OFF seconds with none, again and again. The default is `10,10`, and `--pass 0` removes the gap. The other sensors always send at their rates

With no `--rate` and no `--burst`, the sensor sends at its own rate. A seed fixes the contacts, not the time: each line gets its timestamp from the clock.

Profile	The feed
`ideal`	The default when you give no `--profile`. Each position is random in the area of the sensor. Each line is correct
`world`	One sky that all four sensors see. Where two areas overlap, two sensors report the same object from two positions. `make lab1` uses this profile
`dirty`	The same as `ideal`, with some damaged lines: cut short, too long, empty, or with a field that is not a number
`field`	Contacts that move: the next report about a contact is at its new position
`threat`	`field`, with traffic that filed a plan and a few contacts that filed nothing. A report never says which is which

The environment variable `SENTINEL_PROFILE` has the same effect as `--profile`, and `SENTINEL_SAT_PASS` has the same effect as `--pass`. The other sensors ignore `SENTINEL_SAT_PASS`. An option on the command line has priority over a variable. `SENTINEL_SEED` selects your own set of contacts, which repeats each time.

3.2 The bench programs

The bench is a map box with four real sensors on one board, which the course runs. The bench programs bring its reports to your computer, or simulate the board.

Program	What it does
`benchfeed --sensor NWS|AUR|AIS|SAT`	Writes the bench reports of one sensor on its standard output, as `contactfeed` does. It accepts the other options of `contactfeed`, and ignores them
`benchfeed --health PATH`	Writes the state of the bench to `PATH` each second, for the Bench view[5]
`benchfeed --pty PATH`	Makes a pseudo-terminal at `PATH` that carries each line of the bench
`benchsim --link PATH`	A simulated board behind a pseudo-terminal at `PATH`. `--scenario` selects `empty`, `target`, `strait`, `flood` or `demo`
`benchd --device PATH`	Reads a board, writes its state for the Bench view, and sends its lines to each `benchfeed` that connects, on port 4760 by default

`benchfeed` gets its lines from `--source`, or from the environment variable `SENTINEL_BENCH`: `tcp:HOST:PORT` for a `benchd`, or `file:PATH` for a recorded capture. Its exit status is 0 when its standard output closes or the capture ends. It is 2 when the source is wrong, and 3 when no heartbeat arrives for 3 seconds.

4 The make targets

Run each target at the top of your repository.

Command	What it does
`make`	Builds the given code, copies the given programs, and builds each lab in `src/` that has a `Makefile`
`make test`	Runs the tests of each lab in `src/` that has a `Makefile`. `LAB=2` runs the tests of Labs 1 and 2 only: the stubs of a later lab fail its tests
`make lab1`	Makes a new feed from the four sensors, and runs your Lab 1 demo[6] on it. `PROFILE=dirty` selects a different profile
`make accept1`	Checks your Lab 1 library and demo against their contract[7]
`make watch`	Starts the server of the page at `http://localhost:8080/`
`make submit`	Builds and tests your work, then writes the zip that you upload. The Submission Guide[8] tells you how
`make bench-sim`	A simulated bench board on this computer, for the Bench view
`make bench-health`	The bench board of the course, for the Bench view: `BENCH=tcp:<address>:4760`
`make clean`	Removes `build/`

[5] The page is the operations centre display. Start it with `make watch`, and open `http://localhost:8080/`. `web/README.md`, in your repository, describes the page and each view.

[6] `demo_lab1` is the demonstration program of Lab 1. It reads a feed, a file of contact reports, and writes the mission log. The Lab 1 Brief, Section 3, `docs/lab1/lab1_brief.pdf`, specifies it.

The brief of a later lab names the targets that the lab adds.

5 The page

The page shows the operations centre from the files that your programs write. It only reads files, and it changes nothing. No mark depends on it. `web/README.md` tells you how to start it, how to use it over SSH, what each view shows, and what each lab adds to it.

6 How you work

- Commit often. You can go back to a commit, but the undo of an editor is easy to lose.
- Get each new lab before you start it. The Setup Guide[9] gives the command.
- Record each fault in `docs/DEFECTS.md` when you find it: what you saw, the cause, and the fix. It has no marks, but it is the best preparation for the oral exam.[10]

[7] The acceptance test of Lab 1, `make accept1`, runs 34 checks of your library and your demo. The Lab 1 Brief, Section 4.3, `docs/lab1/lab1_brief.pdf`, describes it.

[8] The Submission Guide, `docs/submit/submit.pdf` in your repository. It gives the marks of each lab, the oral exam, and the steps of the hand-in.

[9] The Setup Guide, `docs/setup/setup.pdf` in your repository. It sets up your machine and your own copy of the repository.

[10] The oral exam of a lab is in the session of the next lab: for Lab 1, the Lab 2 session. The Submission Guide, Section 3, `docs/submit/submit.pdf`, describes it.